

PANINIAN PRATYAHARA ENGINE — CODE LOGIC

What this program does: It takes a proposed Pratyahara, determines the sounds covered by the Iva-Sutra order, and separately checks whether that notation belongs to the configured inventory of attested Paninian Pratyaharas. The code deliberately keeps these two questions separate.

1. The two questions at the heart of the program

RANGE: “If I start at this sound and stop at this anubandha, which sounds lie in between?” This is a mechanical computation from the ordered Iva Sstras.

VALIDITY: “Is this start + marker combination actually recognized as a Pratyahara in the chosen inventory?” The code does *not* infer this merely from the fact that a range can be constructed. This distinction is explicitly built into the program.

■filecite■turn0file0■L8-L13■

Simple mental model:

INPUT → find START → find END MARKER → collect sounds → check INVENTORY → return / reject

2. Build the Iva-Sutra data

The 14 Iva Sstras are stored in order. Every entry contains two pieces of information: the symbol itself and whether it is an **anubandha marker**. For example, an ordinary sound is stored as “not anubandha”, while a marker such as ■■ is marked as an anubandha. The source code also contains a correction to the 11th sstra so that its full sequence is retained.

■filecite■turn0file0■L19-L23■

Why this matters: the program cannot calculate a range unless it knows the exact global order of all symbols and which symbols are stopping markers.

PSEUDOCODE

```
STORE the 14 Iva Sstras in exact order
FOR every symbol: store (symbol, is_anubandha)
CREATE one continuous SEQUENCE containing every item
ALSO remember its sstra number and position
```

3. Convert the nested sstras into one searchable sequence

The program walks through each sstra and each item inside it, then creates one flat sequence. Each item remembers: **symbol**, **is_anubandha**, **sutra number**, and **position inside that sstra**. This makes later searching straightforward.

■filecite■turn0file0■L47-L54■

Conceptually:

```
Sstra 1 → items → Sstra 2 → items → ... → Sstra 14 → items
↓
ONE ordered sequence
```

4. Store the grammatical validity inventory

A second data structure contains the Pratyaharas that the chosen convention recognizes. Some entries include “:N”, for example ■■■:6, because the same marker symbol can occur more than once. The code treats this as an explicit choice of which occurrence of the marker is intended. ■filecite■turn0file0■L59-L68■

There are three modes conceptually:

- **ashtadhyayi** → use the configured attested inventory.
- **custom** → user can provide another inventory.
- **mechanical** → no validity filtering; every constructible pair is allowed, mainly for diagnostics. ■filecite■turn0file0■L86-L89■

5. Prepare the engine

When the engine starts, it scans the sequence and creates two useful collections: unique actual sounds and all anubandhas. Then it selects the requested convention. This means later functions can quickly answer “is this a sound?” and “is this a marker?”.

■filecite■turn0file0■L106-L117■

6. Normalize and parse user input

The input may be written as one notation such as ■■■, or as separate arguments such as ■ + ■■. The program also accepts traditional spelling where the virama is omitted from the first sound. It first normalizes the sound and marker, then splits the notation into (**start sound**, **ending marker**). ■filecite■turn0file0■L147-L188■

PSEUDOCODE

```
IF separate marker is supplied:
  normalize the start sound
  normalize the marker
ELSE:
  remove optional “:sutra” sense
  identify the start sound
  identify the ending marker
IF parsing fails → report invalid input
```

PANINIAN PRATYAHARA ENGINE — LOGIC FLOW

7. Mechanical range calculation

This is the core computation. First locate the starting sound in the flat sequence. Then scan forward until the requested anubandha is encountered. If an explicit s■tra number was supplied, the marker must occur in that s■tra. Finally, collect only non-anubandha symbols and remove duplicates. ■filecite■turn0file0■L190-L221■

PSEUDOCODE

```
FIND index of START in SEQUENCE
IF start does not exist → error

SCAN forward from START
IF item is the requested marker AND (no s■tra specified OR correct s■tra):
  stop at this item
IF marker is never found → error

RESULT = empty list
FOR every item before the ending marker:
  IF item is NOT an anubandha AND not already in RESULT:
    add its sound to RESULT
RETURN RESULT
```

Important: this step answers only “what range does this notation mechanically cover?” It does not yet decide whether the notation is grammatically attested.

8. Find the correct occurrence of a repeated marker

A marker can occur more than once. Therefore the program can maintain a list of valid s■tra numbers for the same (start, marker) pair. If the user writes an explicit sense such as ■■■:6, that occurrence is used. If no sense is supplied and there are multiple attested senses, the first attested sense becomes the default. ■filecite■turn0file0■L223-L251■

PSEUDOCODE

```
LOOK UP (START, MARKER) in INVENTORY
IF an explicit end-s■tra was given:
  use that s■tra
ELSE IF the inventory has attested senses:
  use the first attested sense as default
COMPUTE the corresponding range
```

9. The validation step — the most important part

After computing the range, strict mode asks whether the selected ending s■tra is actually present in the inventory for that start + marker pair. If it is not, the program raises **InvalidPratyaharaError**. Therefore a notation can be mechanically constructible but still be rejected as a recognized P■■inian Praty■h■ras. ■filecite■turn0file0■L253-L263■

PSEUDOCODE

```
COMPUTE the mechanical range
IF strict mode AND an inventory exists:
  determine the actual ending s■tra
  LOOK UP attested s■tras for (START, MARKER)
  IF actual s■tra is NOT in attested s■tras:
    REJECT: “mechanically possible, but not recognized”
  ELSE:
    ACCEPT the result
```

10. Generate all recognized Praty■h■ras

The normal generator does **not** blindly generate every start/marker combination. It iterates through the configured inventory, computes the range for every attested entry, and sorts the results according to ■iva-S■tra order. ■filecite■turn0file0■L273-L297■

PSEUDOCODE

```
FOR every (START, MARKER) in VALID INVENTORY:
  FOR every attested ending-s■tra for that pair:
    compute its range
    create a row containing notation + start + marker + s■tra + sounds
  SORT rows by start position and s■tra number
RETURN rows
```

The separate **mechanical generator** deliberately does the opposite: it tries every later anubandha after every possible starting sound. Those results are diagnostic candidates, *not automatically valid Praty■h■ras*. ■filecite■turn0file0■L299-L317■

11. User interaction and final flow

The demo shows known valid examples, range-only examples with strict validation disabled, and examples intentionally rejected in strict mode. The interactive loop then repeatedly accepts a notation, **all**, or **quit**. For a normal notation it prints the sounds and their count; for an invalid notation it prints the error instead. ■filecite■turn0file0■L333-L365■ ■filecite■turn0file0■L369-L405■

COMPLETE LOGIC IN ONE VIEW

```
INPUT
↓
Parse start + marker (+ optional s■tra sense)
↓
Normalize symbols
↓
Find start in ordered ■iva-S■tra sequence
↓
Find correct ending anubandha
↓
Collect actual sounds in the range
↓
RANGE IS NOW COMPUTED
↓
Check configured attested inventory
↓
Valid? → RETURN RESULT
Invalid? → RAISE ERROR
↓
Interactive mode displays result / error
```

One-line understanding: The program first calculates *what a start + marker covers*, and only afterwards checks *whether P■■ini's configured inventory recognizes that notation*. That separation is the central design of the entire engine.